Chapter 7 練習

作答時標明 fixed-work 或 growing-work、worker count、overhead 與 metric。Speedup 是 elapsed-time ratio；efficiency 是 speedup 除以 worker count。

CH07-Q01 Fixed-work Amdahl 與反解

某固定工作 baseline 的 90% 時間可完全平行，10% 必須序列。無 overhead。

1. 計算 $N = 8$ 的 speedup 與 efficiency。
2. 計算 $N \to \infty$ 的上界；說明它與 finite N 的差異。
3. 若 sequential fraction 降為 3%，128 workers 的 speedup 是多少？
4. 對固定 $N = 8$，把 serial fraction 從 10% 降到 5%；另在 serial 仍 10% 時把 N 從 8 增加到 16。分別求 finite speedup。
5. 若要 $N = 100$ 時達到 speedup 90，至少要有多少 parallel time fraction？將結果換成 serial percentage。

CH07-Q02 Fixed overhead 與負載不均

總工作為 10,000 個等成本 work units，另有 fixed launch overhead 10 units。使用 100 workers，理想是每 worker 100 units。實際一個 worker 分到 2% work，其餘 99 workers 平均分剩餘工作。

1. 計算該重 worker、其他 workers 與 total elapsed time。
2. 以 sequential baseline $10000 + 10$ 計算 speedup 與 efficiency。
3. 若重 worker 分到 5%，重算 elapsed time、speedup 與 efficiency。
4. 說明 work stealing 能改善什麼，以及它新增哪兩種 overhead。

CH07-Q03 Strong scaling、weak scaling 與效率

1. 4-core 在固定問題上花 80 s，single-core baseline 是 300 s。求 strong speedup 與 efficiency。
2. 同一程式成長到4倍 problem size，每core work固定；4-core花90 s，single-core對應較小問題花75 s。先求弱擴展效率；再假設同一問題的單核時間隨工作量線性，求 scaled speedup，並明列兩者比較的規模。
3. 使用原書型 addition 成本 t：序列工作固定 10 additions。小問題有 100 個 parallel additions，大問題有 10,000 個。分別計算 $P = 10$ 與 $P = 100$ 的 elapsed time、speedup。
4. 為何小問題從 10 cores 到 100 cores 只從 5.5 倍升到 10 倍，而大問題可升到 91 倍？
5. Strong scaling efficiency 下降是否直接證明有 data race？列出至少三個替代原因。

Chapter 7 練習 （續）

CH07-Q04 SMP、NUMA、message passing 與 publication

1. System U 有單一 physical address space，所有 processors 到 memory latency 相同；System M 每個 node 有 private physical address space。分別分類這兩個系統。
2. 在 SMP 上，兩個 threads 要發布物件，至少要處理哪兩類正確性？只開 cache coherence 是否足夠？
3. 以下是有效 publication 的簡化程式。初始 `data=0`、`ready=0`；只有一個 writer，且除圖示寫入外沒有其他 thread 修改 data 或 ready：

```
data = 42                    // plain data, protected by publication
release store ready = 1      // atomic flag

acquire load ready           // reader repeats until it reads 1
r = data
```

說明 acquire 何時建立 synchronizes-with，以及為何 `r=42`。若 reader 讀到 `ready=0` 後直接讀 data，能否宣稱已被 release/acquire 排序？ 4. 另定義錯誤版本：`data` 與 `ready` 都是 atomic，所有 store/load 皆採 relaxed。Writer依序寫data=42、ready=1；reader讀ready後再讀data。畫出reader得到ready=1、r=0的允許觀察，說明為何atomicity存在卻沒有跨位置同步。若改成plain non-atomic且無其他同步，為何不能只把結果列為0或42？ 5. Private address spaces 的 message passing 為何需要 copy/serialize、route 與 optional ACK？ACK 是否自動修復 application data race？

CH07-Q05 Hardware multithreading

一個 4-wide superscalar core 常因最後一級 cache miss 等待 200 cycles。它要同時服務多條 independent threads。

1. 比較 fine-grained、coarse-grained 與 SMT 對 long memory stall 的處理。
2. Fine-grained multithreading 是否保證單 thread latency 下降？說明。
3. SMT 為何可提高 issue-slot utilization，但可能降低 each thread 的 cache hit rate？
4. 若每 cycle 可 issue 4 operations、實際平均 2.2，而 single-thread ILP 已足夠填滿 4 slots，加入 SMT 的主要效益可能是什麼？還需要量哪些 downside metric？

Chapter 7 練習 （續）

CH07-Q06 SIMD、成本與Amdahl模型

案例A含80個可向量化操作，每個在純量基準中花1 cycle；另20個不可向量化操作，每個花10 cycles。SIMD寬度為8、各lane完全使用，僅將前80個操作加速8倍。忽略額外載入、設定、時脈與功率變化。

1. 分別計算兩類操作的基準時間、加速後時間、可加速的基準時間比例與整體加速比。說明為何80%操作數不是80%時間。
2. 獨立案例B改成全部100個基準操作各花1 cycle，其中80個可作8倍SIMD加速。計算加速比，再假設每 cycle 最多處理6個操作、尾端不足6個仍占1 cycle，計算加速比及8-lane硬體的實際平均使用率。不可沿用案例A的時間比例。
3. 向量ISA用向量長度描述語意，與固定寬度SIMD相比，如何處理尾端元素與不同硬體lane數？
4. 以案例A/B說明SIMD寬度加倍為何不必使整體加速比加倍，並列出此模型尚未計入的成本。

CH07-Q07 Network topology 與完整計數

考慮 4 rows × 4 columns 的 endpoint graph。Mesh 只在相鄰 row/column endpoints 有 link；torus 再加入每列左右 wrap 與每欄上下 wrap。另比較 K_{16} 與 16-port central crossbar。每條 physical link 在本題按 full-duplex 計一次；bisection 指最小 balanced-cut 的跨越 links。

1. 分別由 rows、columns 求 mesh 與 torus 的 edge count。Torus 的 wrap links 有幾條？
2. 求 mesh 的 degree 分布、直徑與 bisection；求 torus 的 degree、直徑與 bisection。
3. 寫出 torus 的 8 條 wrap endpoints，解釋少畫任一條時為何不再是 degree-4 torus。
4. 求 K_{16} 的 total links、degree、diameter 與 bisection links。若每條 link 單向頻寬是 b，寫出單向 bisection bandwidth。
5. Central crossbar 的 endpoint degree 是否等於 K_{16} 的 degree？若否，哪個資源以 $O(N^2)$ 成長？
6. Uniform traffic benchmark 的 bisection bandwidth 為何仍不足以預測 sparse-matrix application？

Chapter 7 練習 （續）

CH07-Q08 Roofline 與階層 bytes

某 multicore 的 sustained compute ceiling 是 $P_C = 200$ GFLOP/s，DRAM bandwidth $B_D = 40$ GB/s。GB 指 10^9 bytes。

1. 推導單層 roof $P \le \min(P_C, IB_D)$ 與 ridge intensity。
2. Kernel A 的 $I = 0.4$ FLOP/byte；Kernel B 的 $I = 12$。分別求 bound 與 bottleneck。
3. 令一階段 $F = 10^9$ FLOP、$P_C = 1000$ GFLOP/s。DRAM 流量 $Q_D = 0.25 \times 10^9$ bytes、$B_D = 100$ GB/s；shared 流量 $Q_S = 4 \times 10^9$ bytes、$B_S = 1000$ GB/s。分別計算 I_D, I_S、兩個 memory bound 與 compute time。
4. 求跨層 upper bound 與 elapsed-time lower bound。列出 DRAM 點 $(4, 400)$、shared 點 $(0.25, 250)$ 與最終 $(4, 250)$，說明 250 不是 shared roof 的 plateau。
5. 若量測只有 30 GFLOP/s，能否只說「低於 roofline」就算完成最佳化？列出至少兩個診斷方向。

參考資料與署名

David A. Patterson、John L. Hennessy，Computer Organization and Design，Revised Fourth Edition，第 7 章。Lin Cing，《計算機組織》手寫筆記；計算機組織與結構（CSO）課程投影片。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study